

Backtracking

Backtracking

- Suppose you have to make a series of decisions, among various choices, where
 - You don't have enough information to know what to choose
 - Each decision leads to a new set of choices
 - Some sequence of choices (possibly more than one) may be a solution to your problem
- **Backtracking** is a methodical way of trying out various sequences of decisions, until you find one that “works”
- Solutions to many combinatorial optimization problems include exhaustive search and Backtracking can be used to reduce the cost involved

Example

Problem:

- Find out all 3-bit binary numbers for which the sum of the 1's is greater than or equal to 2.
 - The only way to solve this problem is to check all the possibilities:
(000, 001, 010,, 111)
 - The 8 possibilities are called the search space of the problem. They can be organized into a tree generally called as **state space tree**.

State space tree

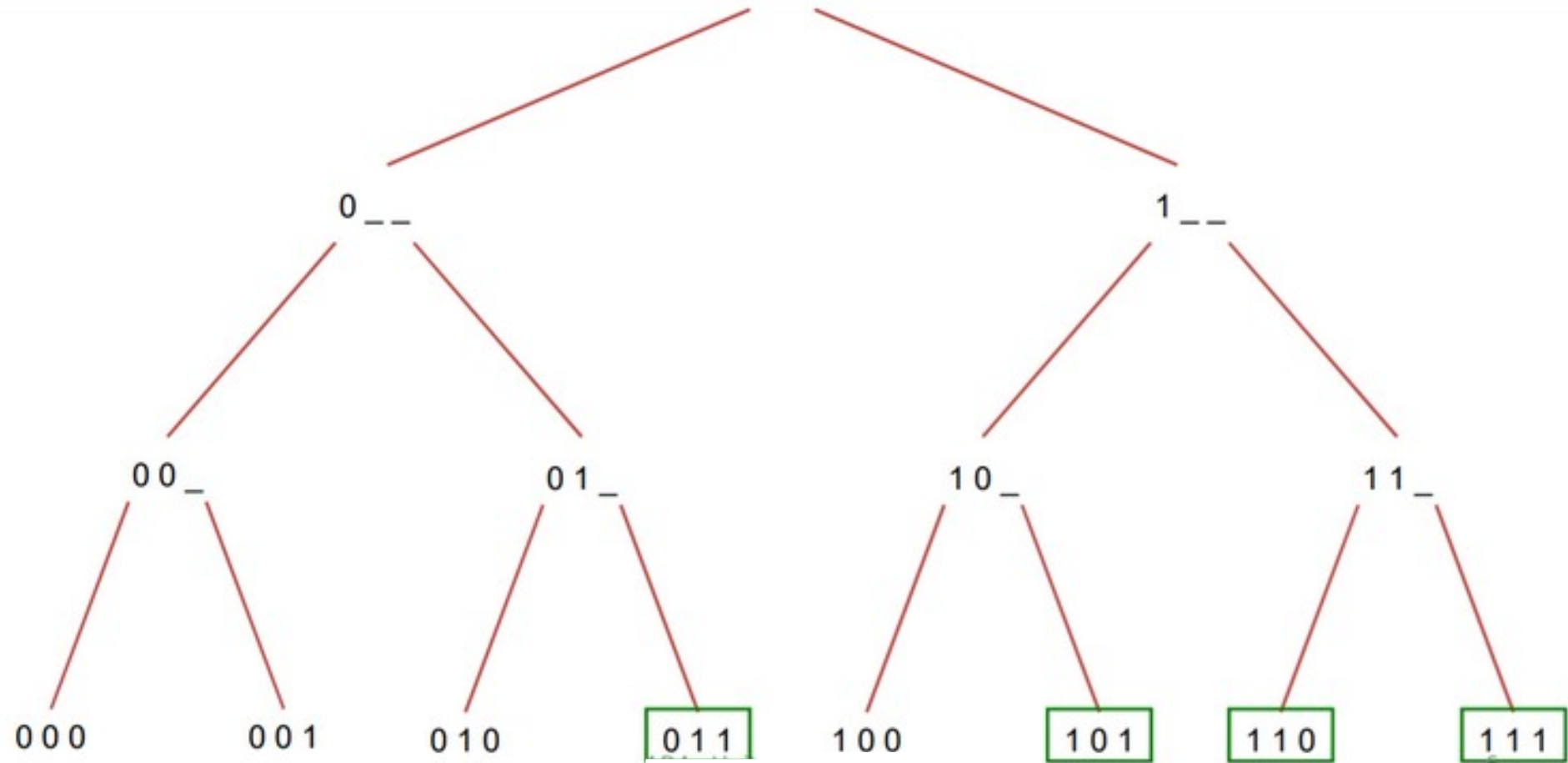
- A rooted tree where each level represents a choice in the solution space that depends on
 - the level above and
 - any possible solution is represented by some path starting out at the root and ending at a leaf
- Root represents state where no partial solution has been made
- A leaf represents the state where all choices making up a solution have been made
- A node in a state-space tree is **promising** if it corresponds to a partial solution that may still lead to a complete solution otherwise it is called **nonpromising**

Coming back to the example...

Problem:

- Find out all 3-bit binary numbers for which the sum of the 1's is greater than or equal to 2.
 - The only way to solve this problem is to check all the possibilities:
(000, 001, 010, ..., 111)
 - The 8 possibilities are called the search space of the problem. They can be organized into a tree generally called as **state space tree**.

Example - state space tree

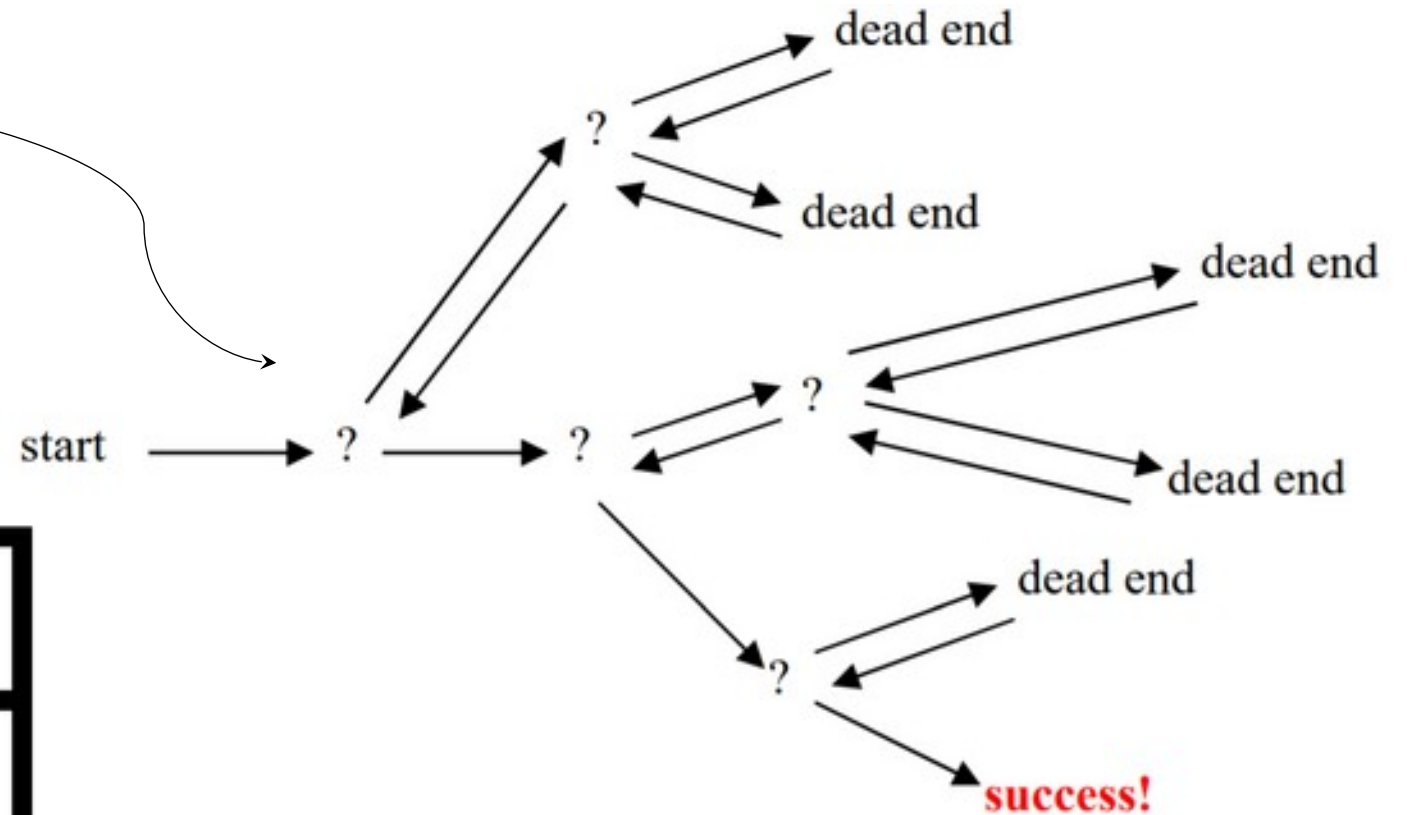
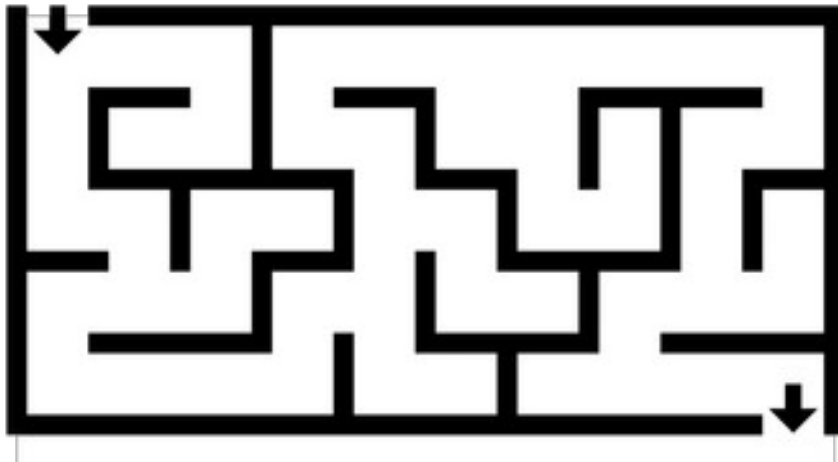


Backtracking - relook

- For some problems, the only way to solve is to check all possibilities.
- Backtracking is a systematic way to go through all the possible configurations of a search space.
- Backtracking is easily implemented with recursion because:
 - The run-time stack takes care of keeping track of the choices that got us to a given point.
 - Upon failure we can get to the previous choice simply by returning a failure code from the recursive call.
- Optimized Backtracking:
 - It help us to reduce the search space and hence get a solution faster.
 - The idea is to avoid those paths that may not lead to solutions as early as possible by finding contradictions so that we can backtrack immediately without the need to build a hopeless solution vector.

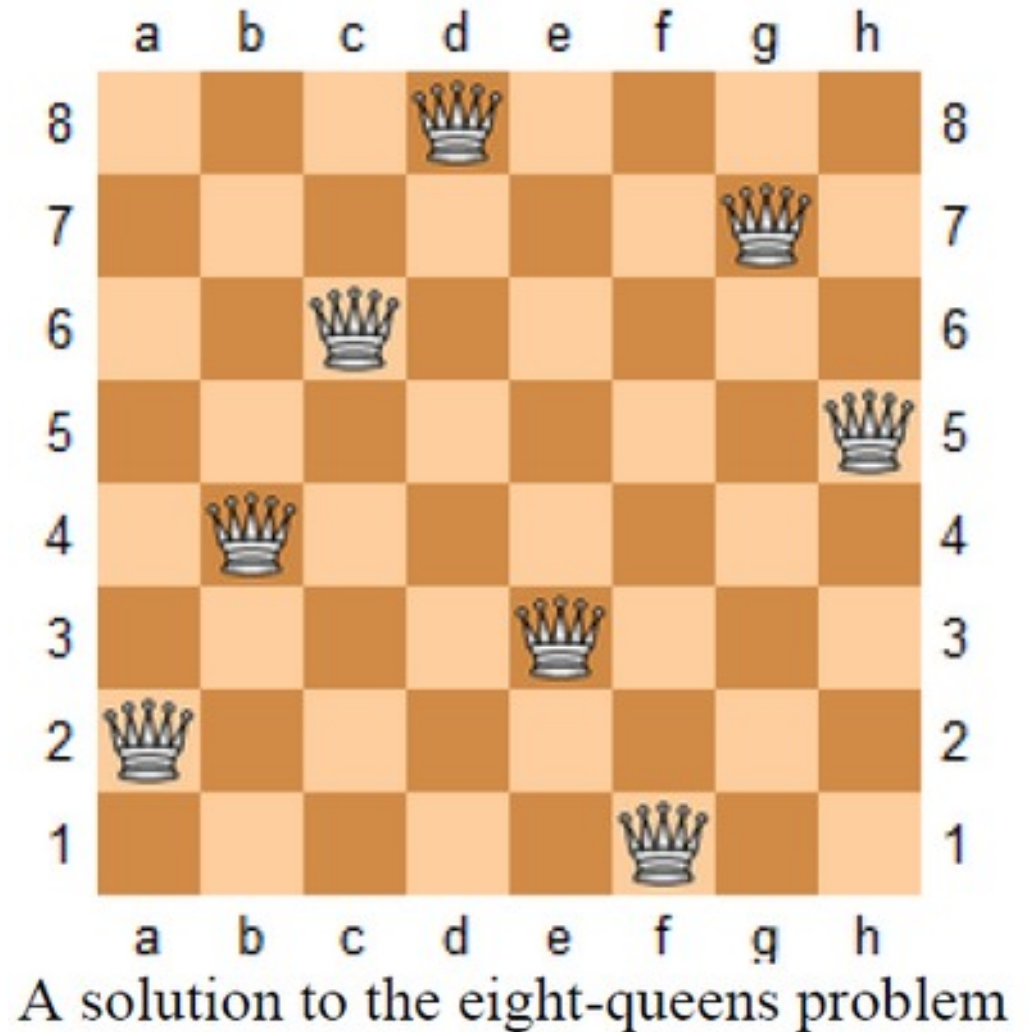
Backtracking Examples

- Solving a Maze
- N Queen's problem
- Sudoku solving
- Map coloring etc.



N-Queen's problem

- Given: n -queens and an $n \times n$ chess board
- Find: A way to place all n queens on the board s.t. no queens are attacking each other (i.e.)
 - No two queens on the same row or column or diagonal
- If $n=1$, trivial case
- No solution for $n=2$ or 3



Solution to 4-queens problem

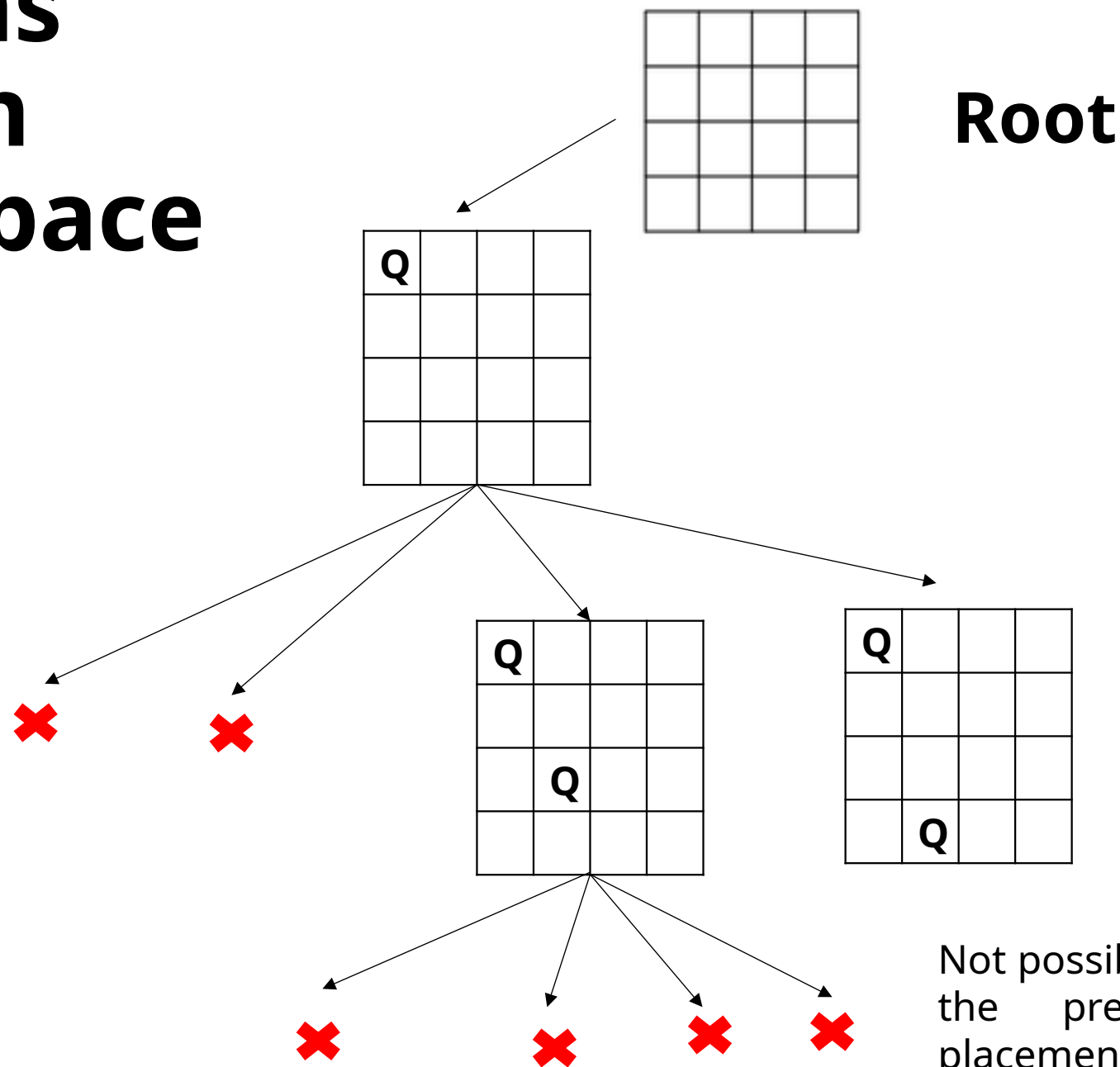
- 1) Start in the leftmost column
- 2) If all queens are placed
return true
- 3) Try all rows in the current column. Do following for every tried row.
 - a) If the queen can be placed safely in this row then mark this [row, column] as part of the solution and recursively check if placing queen here leads to a solution.
 - b) If placing the queen in [row, column] leads to a solution then return true.
 - c) If placing queen doesn't lead to a solution then unmark this [row, column] (Backtrack) and go to step (a) to try other rows.
- 4) If all rows have been tried and nothing worked, return false to trigger backtracking

	Q1	Q2	Q3	Q4
1				
2				
3				
4				

Efficient version is:

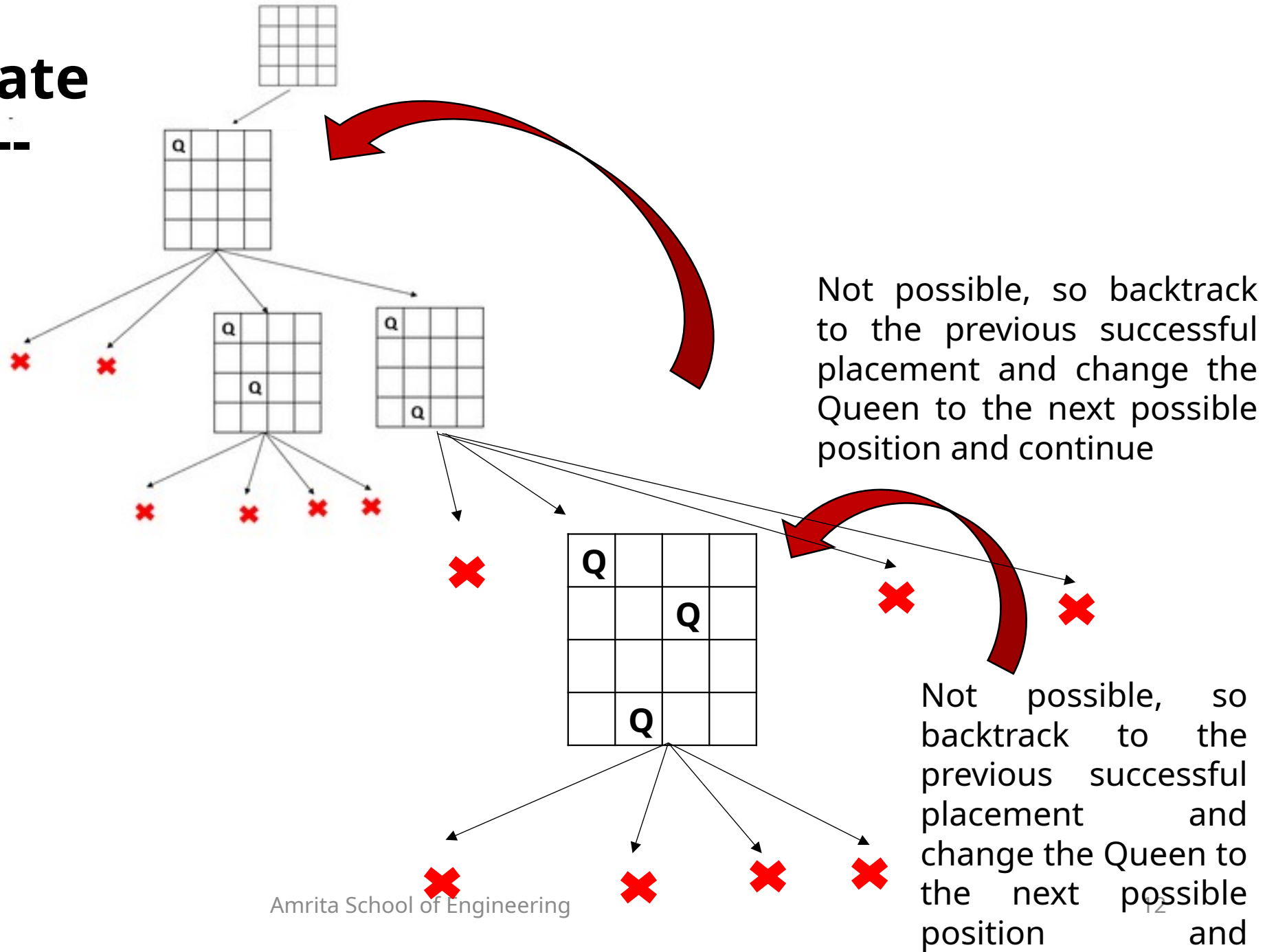
- 1) After placing the first Queen in the first cell, leave the current column and first row in the next column and proceed
- 2) After placing the 2nd Queen, leave the current column and first 2 rows in the next column
- 3) Repeat the same till you have a solution or backtrack to the previous placement and change the position

4-queens problem -state space tree

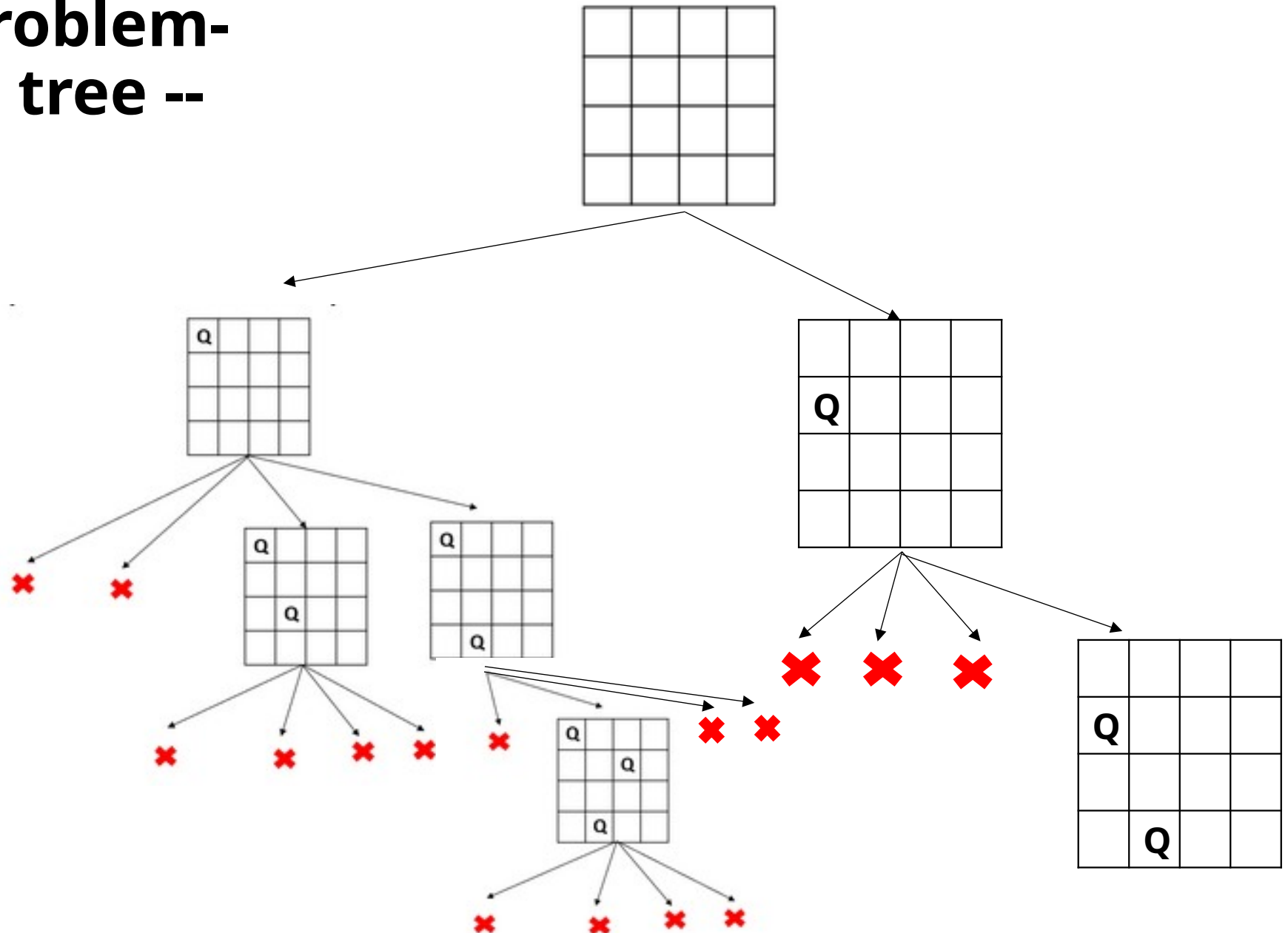


Not possible, so backtrack to the previous successful placement and change the Queen to the next possible position and continue

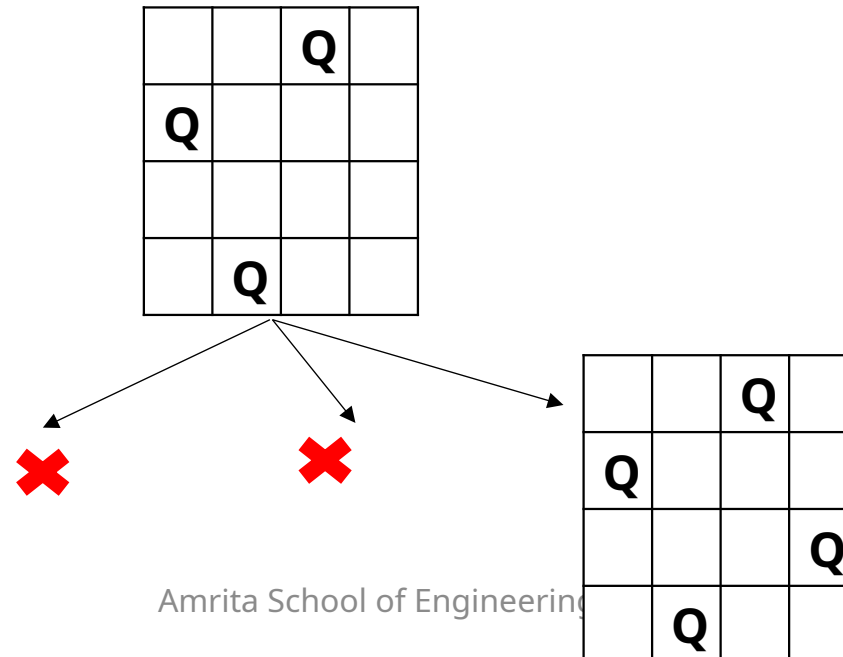
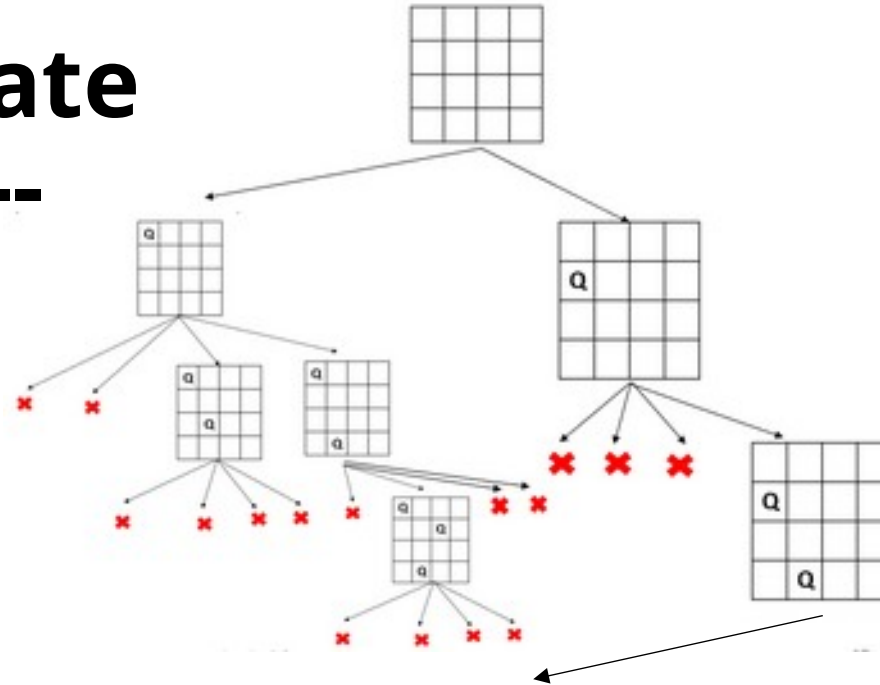
4-queens problem-state space tree -- contd



4-queens problem- state space tree -- contd



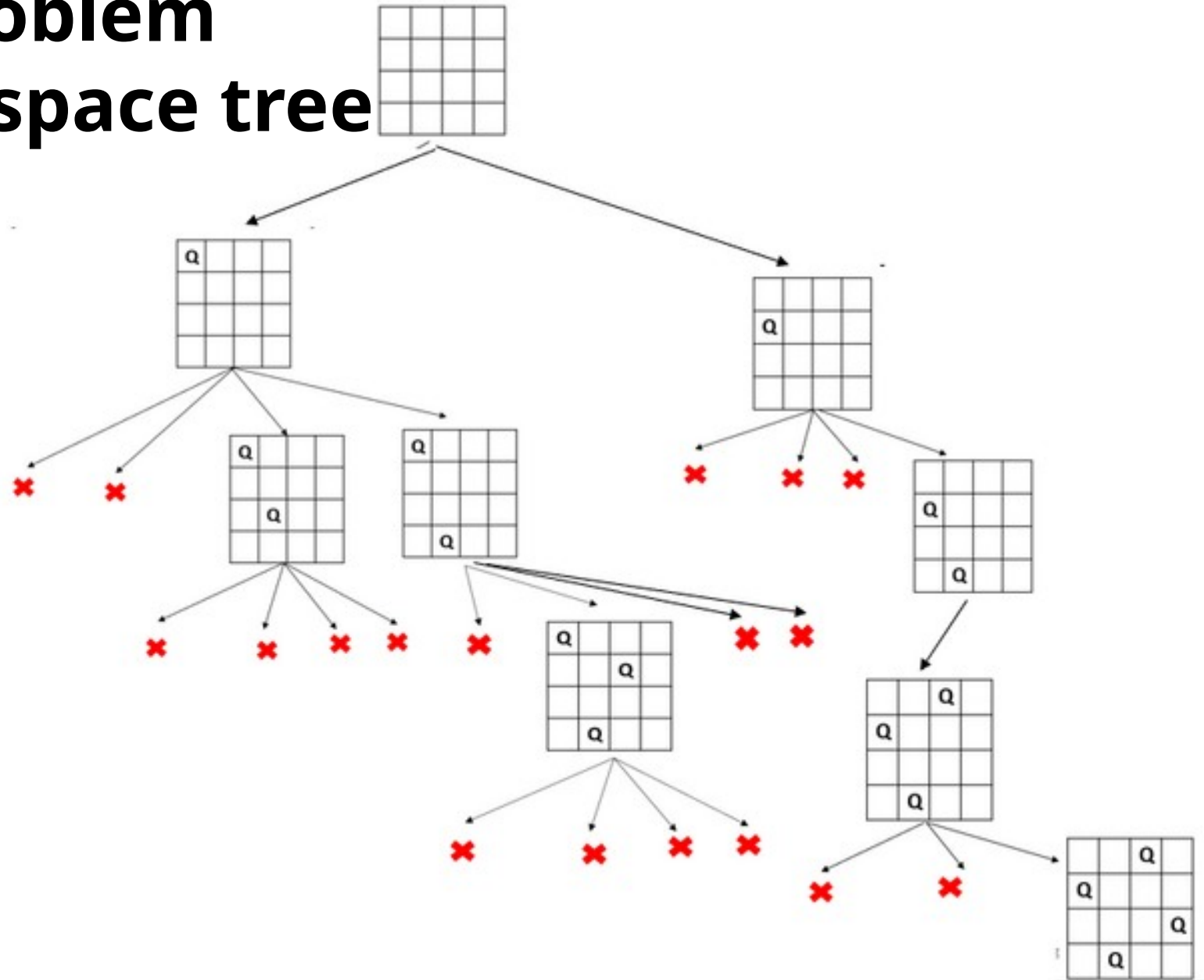
4-queens problem-state space tree -- contd



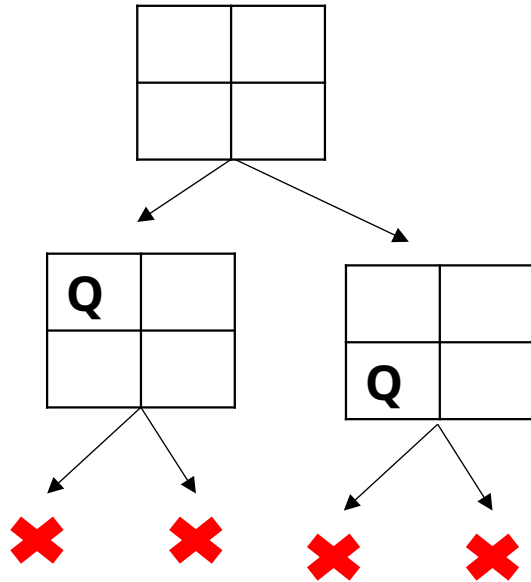
**One Possible
Final
Solution**

4-queens problem

- final state space tree



2-Queens Problem

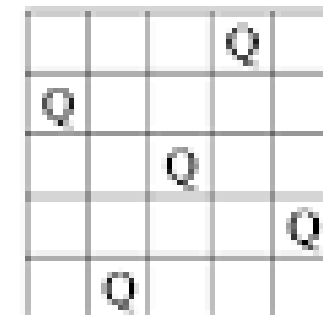
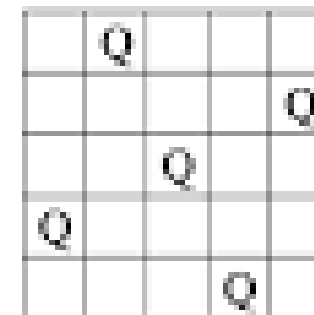
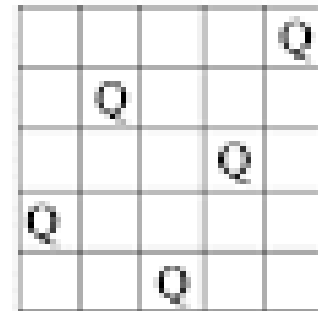
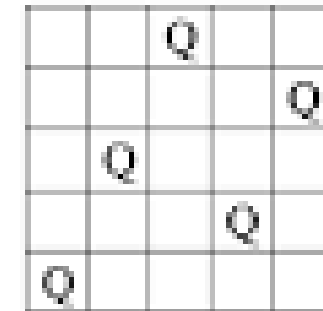
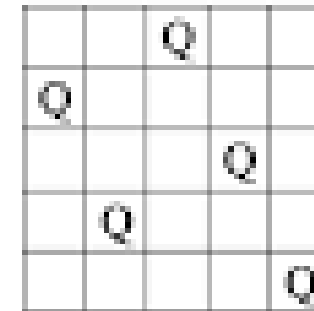
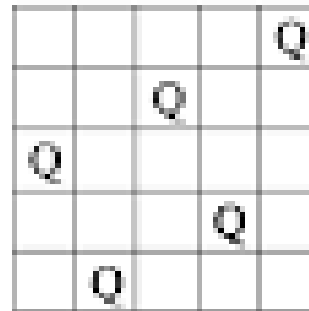
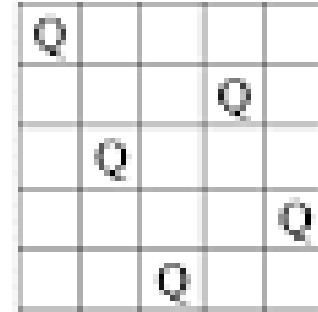
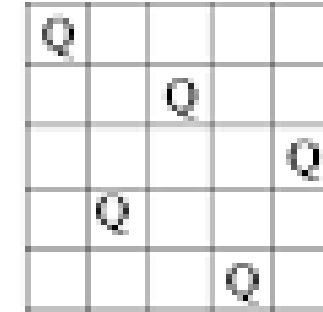
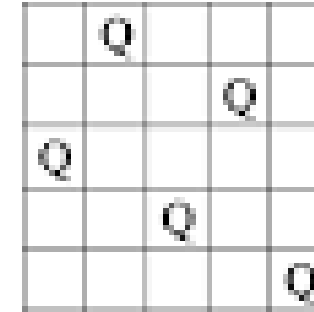
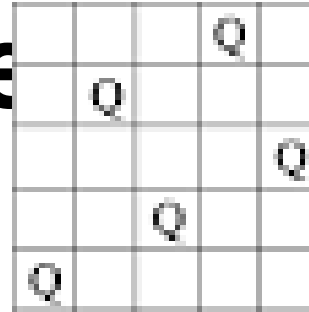


**No
Solution**



5-Queens problem

- Find the solution to place 5 queens on a 5X5 chessboard
- There are many solutions
- Depends on where the first queen is placed



Sudoku

3		6	5		8	4		
5	2							
	8	7					3	1
		3		1			8	
9			8	6	3			5
	5			9		6		
1	3					2	5	
							7	4
		5	2		6	3		

Sudoku Solving - 9x9 grid

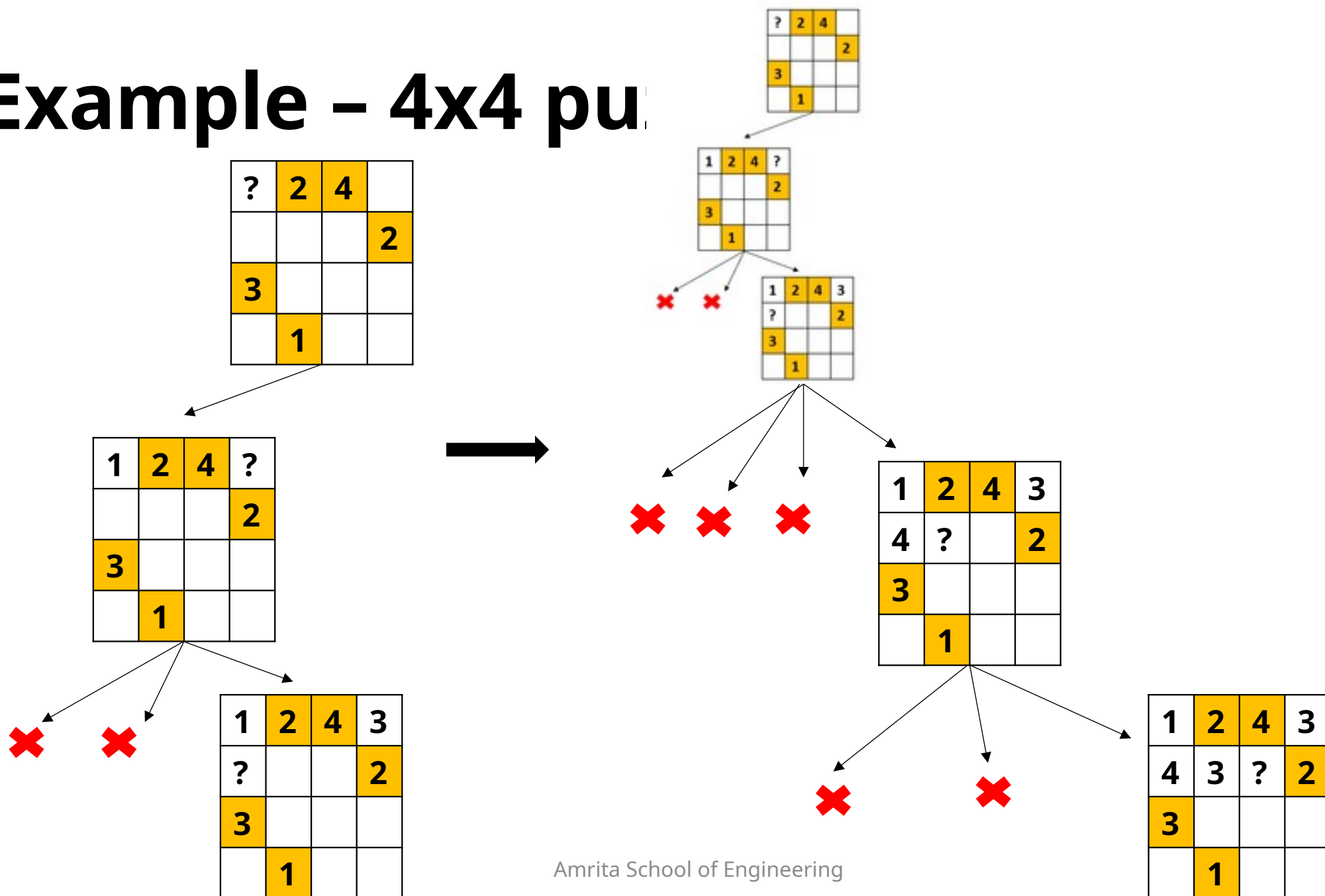
- Find row, column of an unassigned cell
- If there is none, return true
- For digits from 1 to 9
 - a) If there is no conflict for digit at row, col
assign digit to row, col and recursively try filling in the rest of the grid
 - b) If recursion successful, return true
 - c) else, remove digit and try another
- If all digits have been tried and nothing worked, return false

Example - 4x4 puzzle

- Find the missed entries using backtracking approach

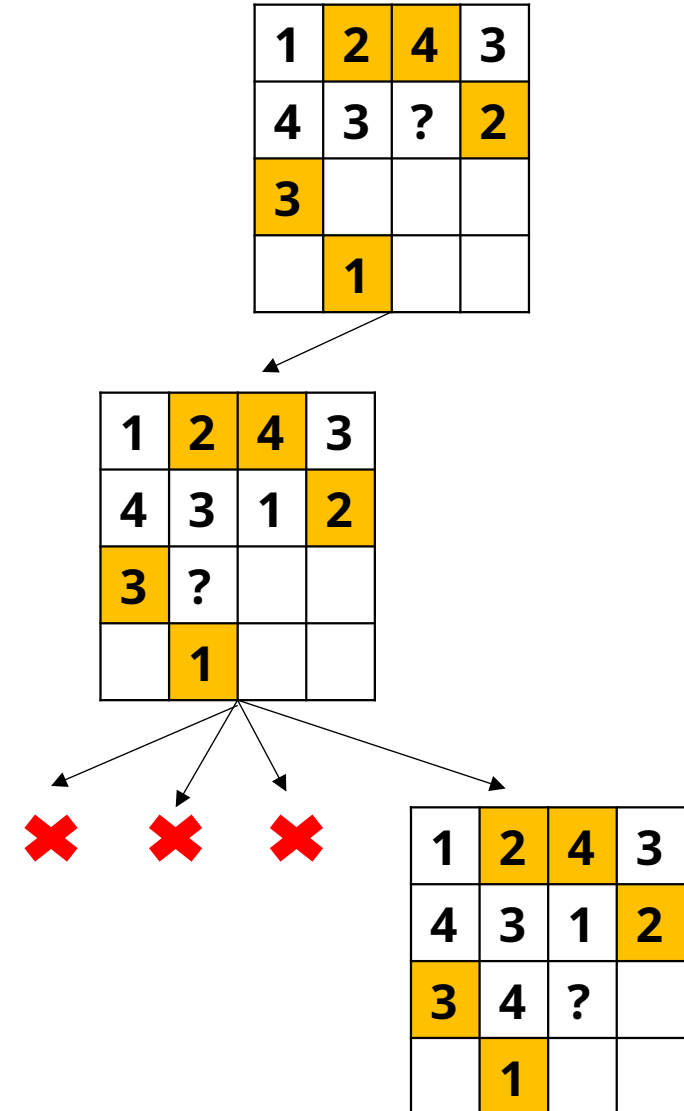
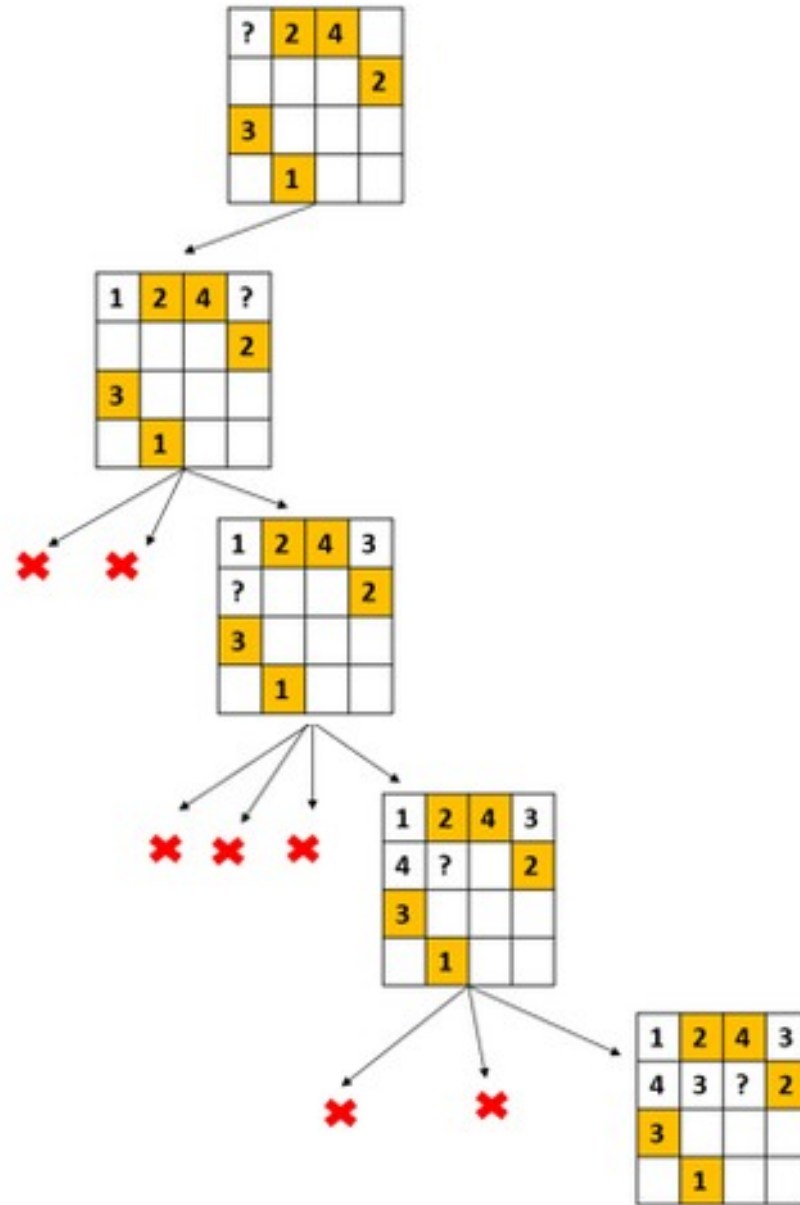
	2	4	
			2
3			
	1		

Example - 4x4 puzzle

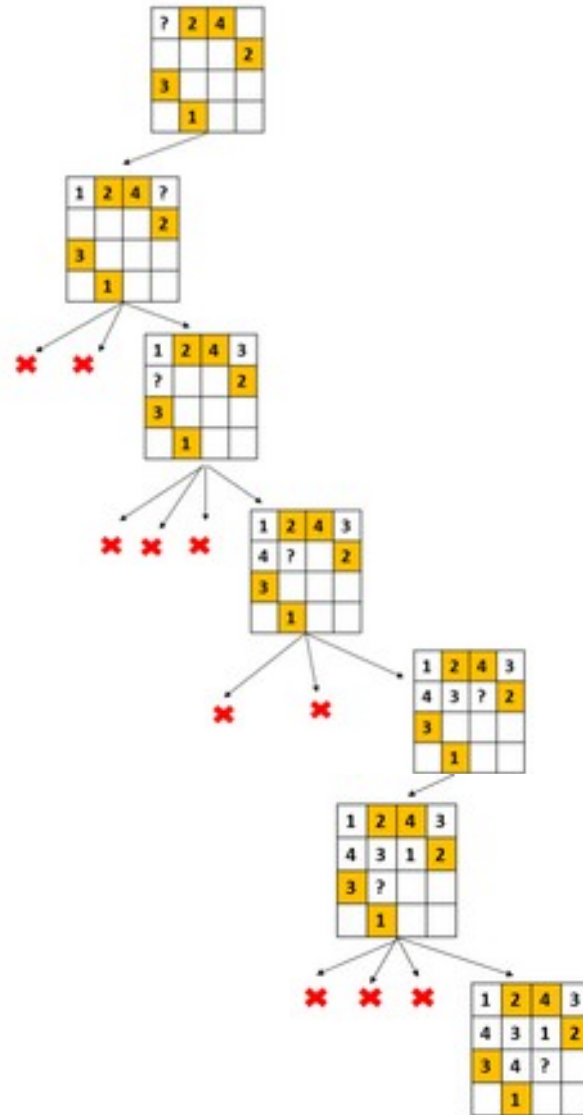


Example

- 4x4 puzzle



Example - 4x4 puzzle



1	2	4	3
4	3	1	2
3	4	?	
	1		

✗

1	2	4	3
4	3	1	2
3	4	2	?
	1		

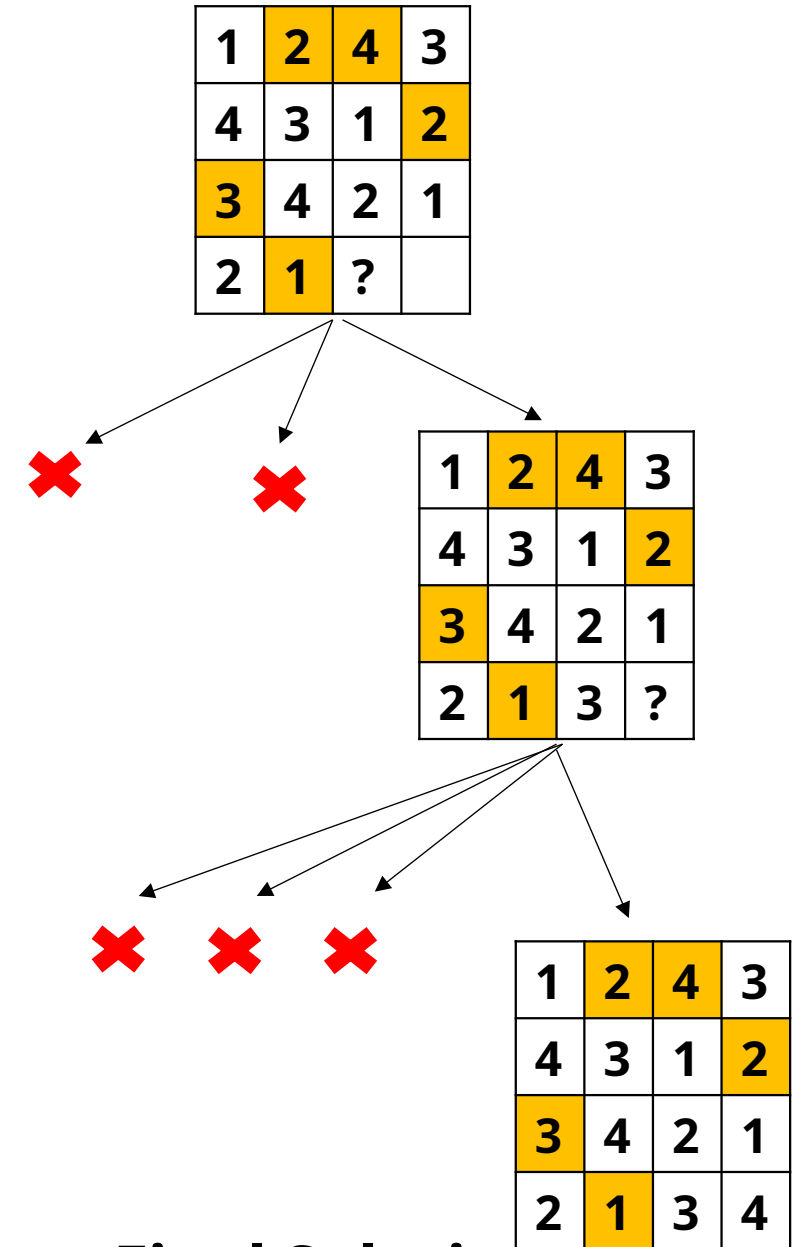
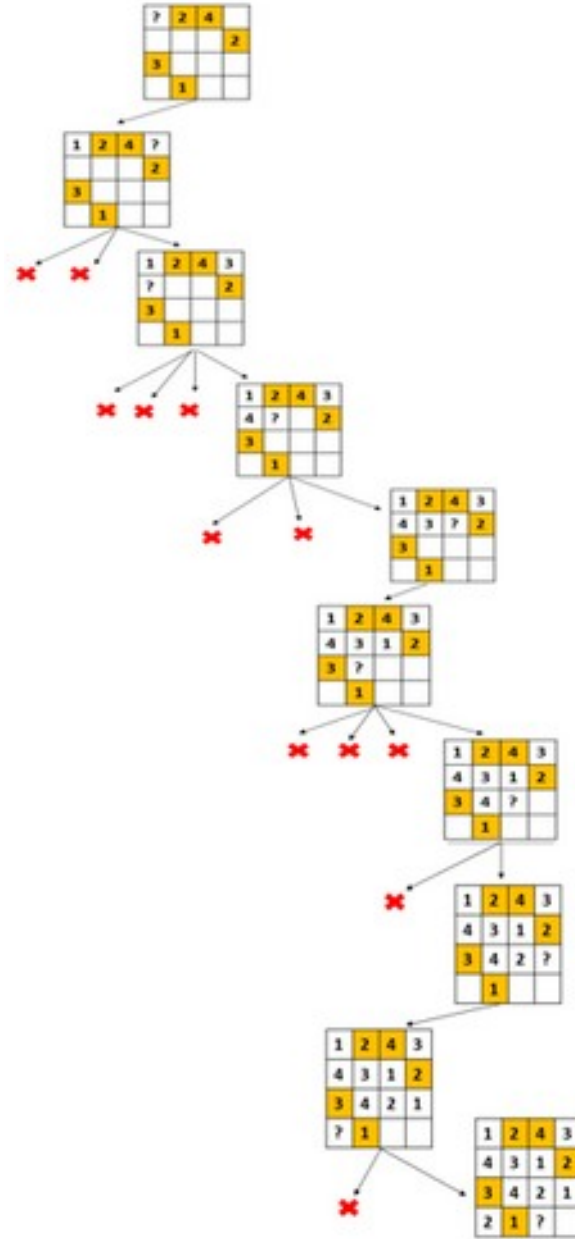
1	2	4	3
4	3	1	2
3	4	2	1
?	1		

✗

1	2	4	3
4	3	1	2
3	4	2	1
2	1	?	

Example

- 4x4 puzzle



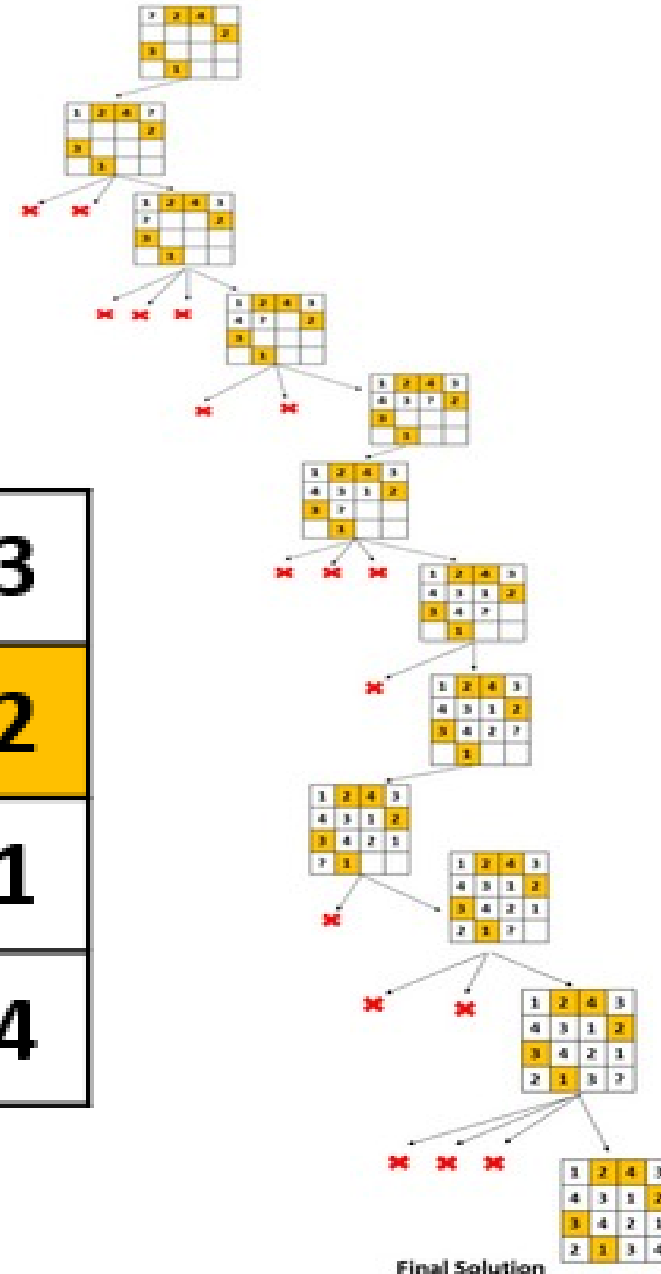
Final Solution

Example - 4x4 puzzle

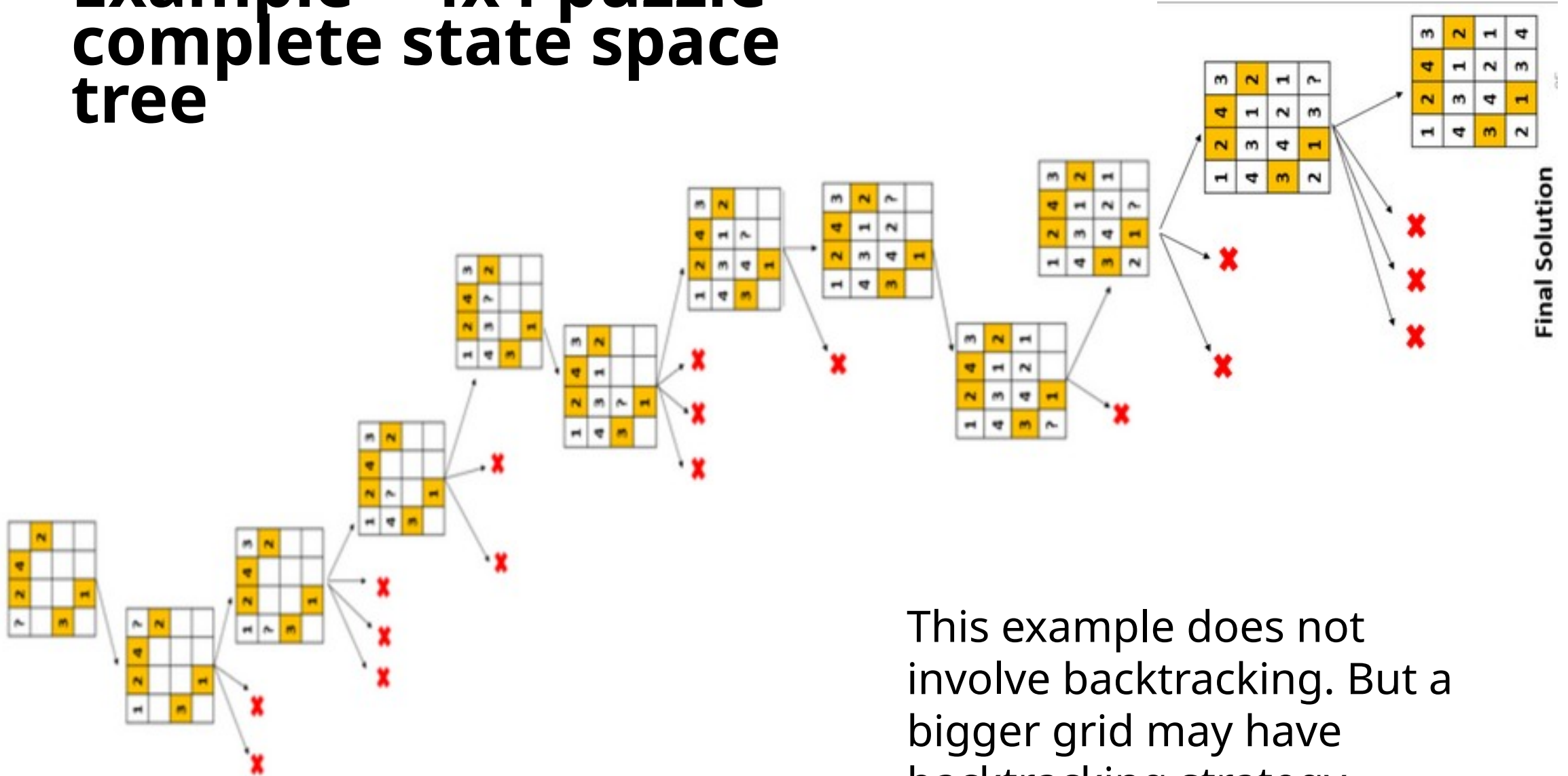
	2	4	
			2
3			
	1		



1	2	4	3
4	3	1	2
3	4	2	1
2	1	3	4



Example - 4x4 puzzle - complete state space tree



This example does not involve backtracking. But a bigger grid may have backtracking strategy applied

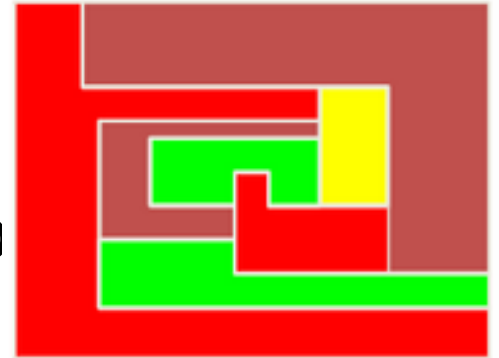
Sudoku – Example 2

- Solve the following puzzle and show the complete state space tree

	1		
4	2		
		2	
	3		

Map/Graph coloring

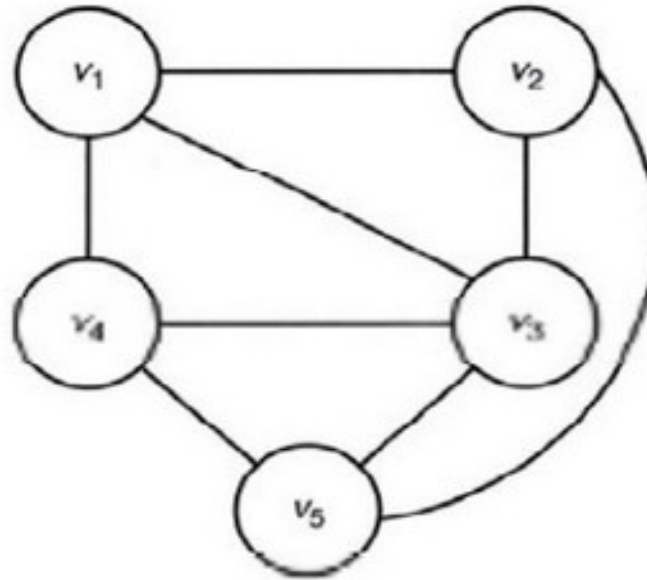
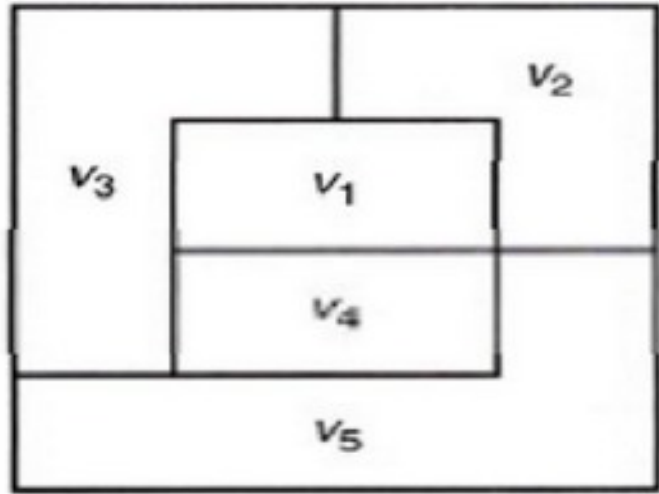
- You wish to color a map with not more than four colors (say..)
 - red, yellow, green, brown
- Adjacent countries/nodes must be in different color
- You don't have enough information to choose color
- Each choice leads to another set of choices
- One or more sequences of choices may (or may not) lead to a solution
- Many coloring problems can be solved with backtracking



Graph Coloring problem - defined

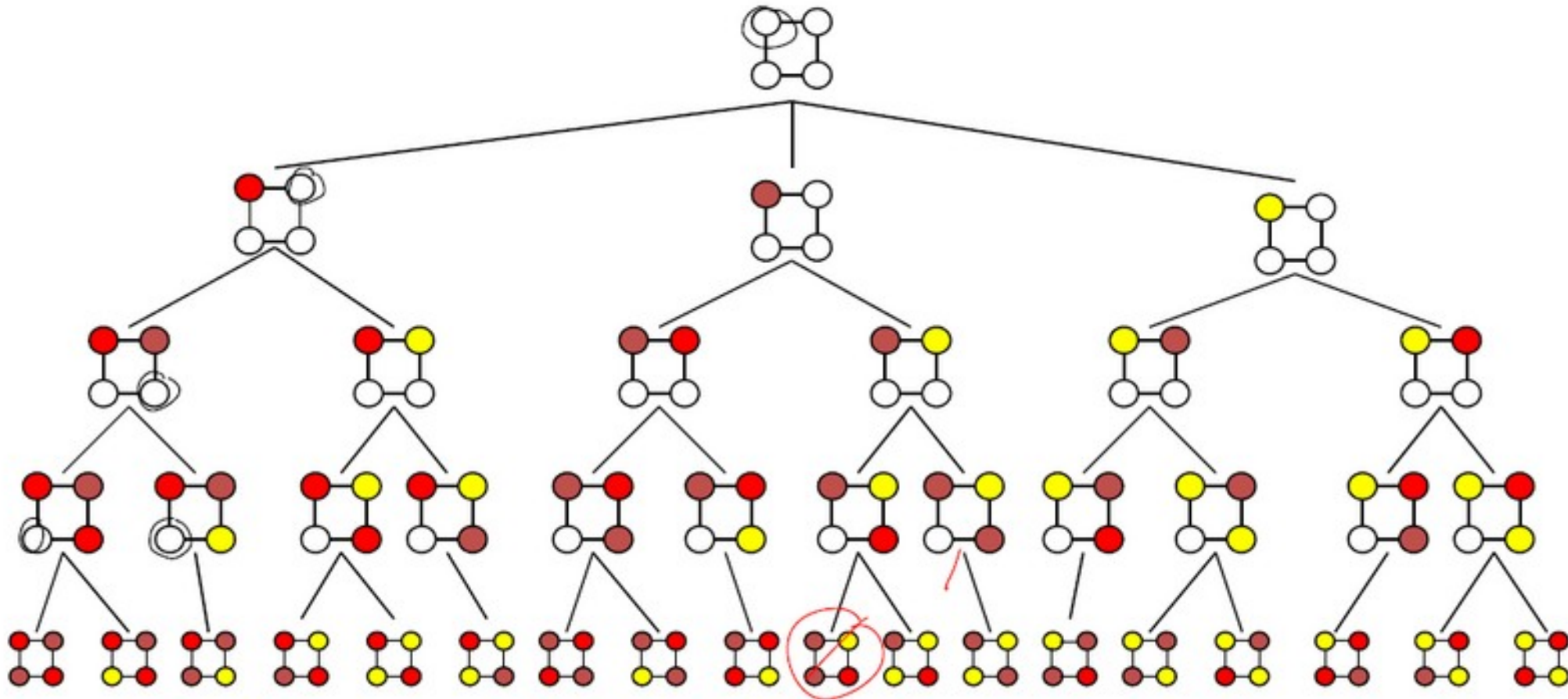
- Finding all ways to color an undirected graph using at most m different colors, so that no two adjacent vertices are the same color.
- Usually the m -Coloring problem consider as a unique problem for each value of m .
- Assign colors to the vertices of a graph so that no adjacent vertices share the same color
 - Vertices i, j are adjacent if there is an edge from vertex i to vertex j .
- Find all m -colorings of a graph
 - Find all ways to color a graph with at most m colors.
 - m is called chromatic number

Map to a Graph



Graph Coloring - Example (state space tree)

- Use 3 colors - red, yellow, brown and find all the possible

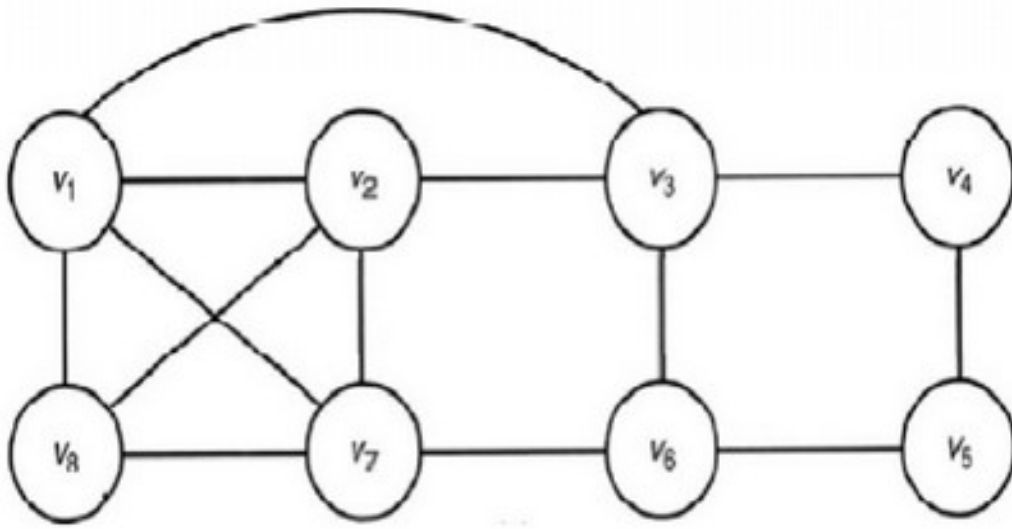


Hamiltonian Cycle/Path

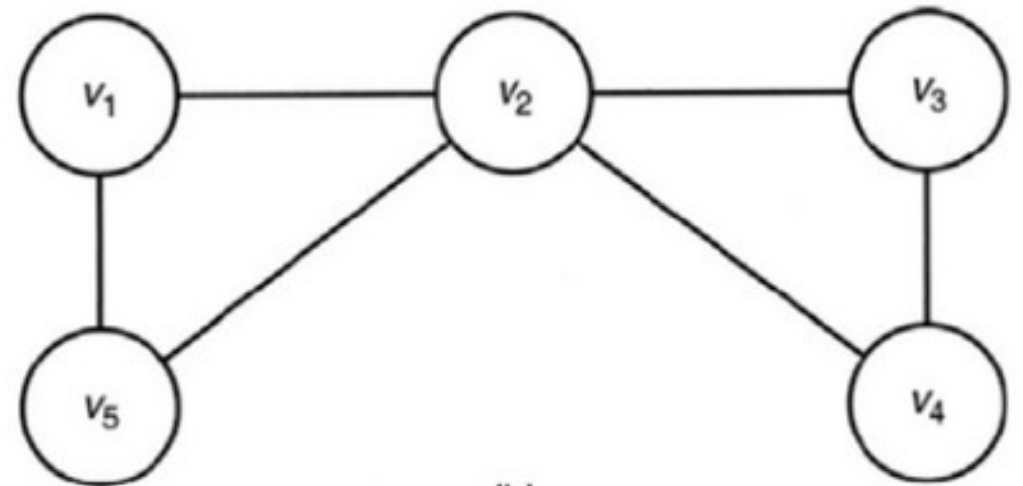
- Hamiltonian cycle (HC): is a cycle which passes once and exactly once through every vertex of G and returns to starting position
- Hamiltonian path: is a path which passes once and exactly once through every vertex of G (G can be digraph).
- A graph is Hamiltonian iff a Hamiltonian cycle(HC) exists.
- This problem can be solved using backtracking approach

Example

- Hamiltonian Circuit
- $[v_1, v_2, v_8, v_7, v_6, v_5, v_4, v_3, v_1]$



No Hamiltonian Circuit!



Complexity

- N-Queen's problem - $O(n^n)$ or $O(n!)$
- Sudoku problem - $O(n^m)$ where n is the number of possibilities for each square (i.e., 9 in classic Sudoku) and m is the number of spaces that are blank
- Solving a Maze (rat in maze) – $O(2^{n^2})$ – n is the size of the maze
- Graph coloring problem - $O(nm^n)$
//where n =no. of vertex, m =no. of color used